Security Assessment Report

AeroxERP x Plouton AI Proposal Landing Page

Date: July 28, 2026

Tested by: M. Maaz Wali Khan, Cybersecurity Intern

Organization: AeroxERP

Live Site Tested: https://plouton-ai-proposal-landingpage-mg7-smoky.vercel.app/

Hosting Platform: Vercel

Scope: Live production landing page (hosted on Vercel) and associated source code repository.

Methodology: Time-boxed initial triage assessment combining source code review (dependency audit, secrets scan) and live-site testing (HTTP header analysis, exposed file checks, manual input testing).

---

1. Executive Summary

A time-boxed security triage was performed on the **AeroxERP x Plouton AI** proposal landing page and its associated source repository. The assessment covered dependency vulnerabilities, hardcoded secrets, HTTP security headers, exposed sensitive files, and basic input handling on the live site.

Overall, the application presents a **Low-Medium** risk posture at this stage. No critical vulnerabilities, exposed secrets, or injectable input points were identified. A small number of Medium and Low severity issues were found, primarily related to missing HTTP security headers. This was a time-boxed initial review; a deeper follow-up assessment is recommended before the site is considered fully production-hardened (see Section 5).

2. Scope and Methodology

- Reviewed source code repository provided via Git access, including dependency manifests and application source files.
- Ran automated dependency vulnerability scan (npm audit) against the project's package manifest.
- Searched source files and Git history for hardcoded credentials, API keys, and tokens.
- Analyzed HTTP response headers from the live Vercel deployment for security header configuration.
- Attempted direct access to commonly sensitive paths (.env, .git/config) on the live deployment.
- Manually tested the contact form for injection behavior and input validation weaknesses.

Note: This was a time-boxed triage assessment, not an exhaustive penetration test. Active scanning, authenticated testing, and business logic testing were out of scope for this pass due to time constraints.

3. Findings

Finding	Severity	Evidence / Location	Recommendation
Missing security headers (Content-Security-Policy, X-Frame-Options, X-Content-Type-Options)	Medium	curl -I response from live site did not include these headers. Only Strict-Transport-Security was present and correctly configured.	Add missing headers via next.config.js headers() function or vercel.json. Start with a baseline CSP such as default-src 'self'.

Wildcard CORS policy (Access-Control-Allow-Origin: *)	Low / Informational	Observed on live site response headers for the tested resource.	Confirm this is intentional for public static content. If any API routes share this policy and return sensitive/user-specific data, restrict to trusted origins.
Weak client-side email validation on contact form	Low	Contact form (mailto-based) allows submission with an empty email field despite an "@" format check.	Add a non-empty validation check in addition to the existing "@" format check.

4. Areas Reviewed With No Issues Found (Frontend / Live Site)

- Dependency Vulnerabilities: npm audit reported 0 known vulnerabilities in current dependencies.
- Hardcoded Secrets: No API keys, passwords, or tokens found in JavaScript source files, JSON files, or accessible Git history.
- Sensitive File Exposure: Direct requests to /.env and /.git/config on the live deployment did not return file contents (redirected / not accessible).
- Cross-Site Scripting (Contact Form): The contact form uses a client-side mailto: link rather than server-side form submission. Test payload <script>alert(1)</script> was rendered as inert plain text in the resulting email body only, with no code execution or reflection on the site itself.
- Transport Security: Strict-Transport-Security header is correctly configured with includeSubDomains and preload directives.

5. Backend Code Review

A folder named "**self-healing-agent-poc"** in the "main" branch of the repository containing a Python/FastAPI backend service was also reviewed. This service exposes an API used for AI-powered UI change detection.

Method: pip-audit dependency scan, manual review of both backend source files (main.py, detector.py), and review of committed demo/test fixture files.

Finding	Severity	Evidence / Location	Recommendation
CORS misconfiguration: wildcard origin combined with credentials allowed	High	main.py: CORSMiddleware configured with allow_origins=["*"] and allow_credentials=True in self-healing-agent-poc/backend.	Set allow_origins to an explicit list of trusted domains. Never combine a wildcard origin with allow_credentials=True, as this allows any website to make credentialed cross-origin requests to the API.
No authentication or rate limiting on /detect endpoint	Medium	main.py: POST /detect endpoint is publicly callable with no auth check. Internally triggers calls to the Gemini API (GEMINI_API_KEY).	Add API key/authentication requirement and/or rate limiting to prevent unauthorized use and unexpected third-party API costs.
No file size limit on file uploads	Low / Medium	main.py: uploaded files are read fully into memory via await file.read() with no size check.	Enforce a maximum content-length before reading uploads to

			prevent memory-exhaustion (denial-of-service) risk.
Unhandled exception on non-UTF-8 file upload	Low	main.py: .decode("utf-8") call on uploaded file content has no error handling.	Wrap decode logic in try/except and return a clean 400 response for invalid input rather than an unhandled server error.

5.1 Backend Areas Reviewed With No Issues Found

- Dependency Vulnerabilities: pip-audit reported 0 known vulnerabilities in requirements.txt.
- Hardcoded Secrets: Credentials (e.g. GEMINI_API_KEY) are correctly referenced via environment variables (os.getenv), not hardcoded. No .env file is tracked in the repository.
- Debug Mode: No debug flags found enabled in reviewed source files.
- Injection Risks: No raw SQL execution, shell command execution, or use of eval/exec found. HTML parsing is handled via BeautifulSoup, which does not process external entities.
- Demo/Test Data: Sample HTML fixture files (old_page.html, new_page.html) used to demonstrate the detection feature contain fabricated placeholder company and vendor data, with no real user data, credentials, or PII.

6. Recommendations

- Priority 1 (High): Fix backend CORS configuration — do not combine allow_origins=["*"] with allow_credentials=True. Restrict to explicit trusted origins.
- Priority 2 (Medium): Add authentication and/or rate limiting to the /detect API endpoint to prevent unauthorized use and unexpected third-party API costs.
- Priority 3 (Medium): Implement missing HTTP security headers (Content-Security-Policy, X-Frame-Options, X-Content-Type-Options) on the live site via next.config.js or vercel.json.
- Priority 4 (Low): Enforce a maximum upload size on the /detect endpoint and handle non-UTF-8 input gracefully.
- Priority 5 (Low): Review and, if appropriate, restrict the frontend's wildcard CORS policy; strengthen contact form email validation to reject empty input.
- Follow-up: Conduct a full penetration test covering authenticated flows, all API endpoints, business logic, and active vulnerability scanning once a staging environment is available, to build on this initial triage.

7. Conclusion

Based on this initial assessment, the **AeroxERP x Plouton AI** proposal landing page does not exhibit critical security weaknesses. The identified issues are low-to-medium severity and are straightforward to remediate. Addressing the missing security headers is recommended before wider distribution of the proposal link.

Prepared by: M. Maaz Wali Khan

Role: Cybersecurity Intern

Date: July 28, 2026